

HeliosStaxV2 & OrxStax Security Review

Conducted by: **Georgi Trachev**

Report Version 1.0
February 26, 2025

Contents

1. About Georgi Trachev

2. Disclaimer

3. About the protocol

4. Risk Classification

4.1. Impact

4.2. Likelihood

4.3. Actions required by severity level

5. Security Assessment Summary

6. Executive Summary

7. Findings

1. About Georgi Trachev

Georgi Trachev is a smart contract security researcher. Throughout his experience he has uncovered numerous critical vulnerabilities in a wide variety of protocols. Reach out on Telegram at <https://t.me/trachevgeorgi>.

2. Disclaimer

Smart contract security reviews are a time, resource and expertise bound effort where security researchers do their utmost to find as many vulnerabilities as they can. They are not a 100% guarantee of the security of smart contracts.

3. About the protocol

HeliosStaxV2 and OrxStax are decentralized protocols deployed on the Ethereum blockchain.

HeliosStaxV2 is a staking protocol for Helios, through which Helios stakers can earn rewards in different tokens. Reward distribution is initiated periodically in 10-day cycles.

OrxStax is a Stax staking protocol for Ouroboros. Accumulated TitanX is swapped for ORX, which is later deposited into an ORX staking protocol. The stake earns rewards in a variety of tokens.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

High: Causes a significant loss of funds, the protocol is directly compromised

Medium: A minor amount of funds at loss, or a functionality of the protocol can be affected

Low: Unexpected behaviour in the protocol, but does not present a significant harm

4.2 Likelihood

High: Direct attack vector; cost of attack negligible, compared to funds at risk

Medium: A sophisticated attack vector that is incentivized only in some conditions but is still likely to occur

Low: Requires many unlikely assumptions or the attacker is not incentivized

4.3 Actions required by security level

High: Issue **MUST** be fixed

Medium: Issue **SHOULD** be fixed

Low: Issue **MAY** be fixed

5. Security Assessment Summary

Overview

Repository: <https://github.com/DudeGuy420/HLX-Staking-v2>

Commit hash: 53d65b71a1134bbb39c67e2bc200e6ba78cb01f3

Repository:

<https://github.com/DudeGuy420/Stax/tree/main/contracts/OrxStax>

Commit hash: c9a4ac0a7a361719c01de6af43562d3b4bb639da

Scope

The smart contracts included in the scope of the security review:

- HLX-Staking-v2/contracts/HeliosStaxV2/HeliosStaxInstanceV2.sol
- HLX-Staking-v2/contracts/HeliosStaxV2/HeliosStaxV2.sol
- HLX-Staking-v2/contracts/HeliosStakingV2.sol
- Stax/contracts/OrxStax/OrxStax.sol

Timeline

Audit start: February 20, 2025

Preliminary report: February 26, 2025

6. Executive Summary

Throughout the security review, conducted in the period February 20-26, 2025, 6 issues were discovered.

Findings count

High Risk	1
Medium Risk	1

Low Risk	3
Informational	1

7. Findings

7.1 High Findings.

[H1] Users can completely block HeliosStaxInstanceV2 from claiming HeliosStakingV2 rewards

Severity: *High*

Context: HeliosStaxInstanceV2.claimRewardsV2.sol#L52

Description: In HeliosStaxInstanceV2 users can claim HeliosStakingV2 rewards by calling *claimRewardsV2*, which makes an external call to *HeliosStakingV2.claimRewards*. In order for the call to be successful the *stakeIds* provided must have a length equal to exactly *totalStakes* - *_totalEndedStakes*. *_totalEndedStakes* is updated only in *endStakeAfterMaturity* which any user can call in order to end a Helios stake after it has matured.

The issue is that Helios provides a second way for ending a mature stake: the *endStakeForOthers* function. It works the same way as the normal *endStake* function, however, it allows users to end the stake of other addresses. Therefore, any user can call *Helios.endStakeForOthers* on a matured stake made by HeliosStaxInstanceV2.

This will be extremely problematic as even though the stake was ended *_totalEndedStakes* was not updated. Now any call to

HeliosStaxInstanceV2.claimRewardsV2 will revert as callers will be forced to provide *stakeIds* with a length that is higher than the actual active stakes. Ultimately, this will cause *HeliosStakingV2.claimRewards* to revert as it does not allow claiming rewards for stakes that have been ended.

Recommendation: There are two ways the issue can be mitigated:

1/ Include an admin function that allows updating *_totalEndedStakes* in the case that the issue has occurred.

2/ Instead of requiring users to provide the *stakeIds* in *HeliosStaxInstanceV2.claimRewardsV2* the stake Ids can be queried from *Helios.getUserStakes* and then ended stakes can be filtered out in another for loop, however, this mitigation will use much more gas.

Resolution: Resolved.

7.2 Medium Findings

[M1] *HeliosStaxInstanceV2* may miss out on *HeliosStakingV2* cycle rewards

Severity: *Medium*

Context: *HeliosStakingV2.claimRewards.sol#L146*

Description: *HeliosStakingV2* distributes rewards based on 10-day cycles, if a user does not claim their distribution of the rewards before the cycle has ended their rewards will be used for the distribution of the next cycle, thus they will ultimately lose them.

The issue is that it is highly likely for *HeliosStaxInstanceV2* to miss out on claiming its share of the rewards due to the way claiming V2 rewards is handled. In order for claiming V2 rewards to be successful the *stakeIds* provided in the call must have a length equal to exactly *totalStakes - _totalEndedStakes*, which are the active stakes. The issue is that it is possible for the stakes to have been made in different

cycles, which is problematic as it will prevent claiming rewards for the earlier cycles. For example:

1/ The current timestamp is $T1$, In HeliosStakingV2 a new cycle will begin in $T2$, thus every stake, which was made before $T2$ will be eligible to earn a share of the rewards.

2/ A HeliosStaxInstanceV2 stake, which we will call $S1$, is made in $T1$.

3/ The current timestamp increases to $T2$, thus $S1$ can earn a share of the rewards.

4/ However, before *claimRewardsV2* is called another stake is made: $S2$.

5/ Now if users attempt to claim the V2 rewards for $T1$ they will be unable to do so as they must provide both $S1$ and $S2$ as the *stakeIds* in *claimRewardsV2*. However, this will cause *HeliosStaking.claimRewards* to revert as $S2$ was made after the current *_lastCycleT*.

Recommendation: HeliosStakingV2 should skip ineligible stakes instead of reverting.

Resolution: Resolved.

7.3 Low Findings

[L1] HeliosStaxInstanceV2 may miss out on HeliosStakingV2 cycle rewards when matured stakes are ended

Severity: *Low*

Context: HeliosStaxInstanceV2.endStakeAfterMaturity.sol#L56

Description: When users end matured HeliosStaxInstanceV2 stakes, v2 rewards for the stakes may have not been claimed. In that case the

protocol will lose the cycle rewards for that stake as HeliosStakingV2 does not allow claiming rewards for ended stakes.

Recommendation: One solution is to make a call to *HeliosStakingV2.claimRewards* before ending the stakes. This, however, will be gas intensive and will likely revert if rewards have already been claimed.

Resolution: Acknowledged.

[L2] HeliosStakingV2 rewards may be lower than intended

Severity: *Low*

Context: HeliosStakingV2.updateCycle.sol#L98

Description: In HeliosStakingV2 if a stake was made in the same block as the call to *updateCycle* its shares will be included to the reward allocation calculations, however, the staker will not be eligible to earn a share of the rewards. Thus, their shares will simply reduce the reward allocations for the other users.

Recommendation: There is no effective solution to the issue which will not cause further problems and the likelihood of the issue occurring is extremely small, thus I do not believe there is a need for mitigation.

Resolution: Acknowledged.

[L3] Users may be unable to claim their cycle rewards if a user gets unblacklisted

Severity: *Low*

Context: HeliosStakingV2.setBlacklisted.sol#L192

Description: In HeliosStakingV2 if a user gets blacklisted their stakes are not included when calculating the reward token allocations and they are not eligible to earn rewards.

The issue is that if a user has been blacklisted and after that unblacklisted they will be eligible to earn a portion of the cycle rewards, even though their stake shares were not included in the token allocation calculations. As a result, there will not be enough reward tokens in the contract, thus if the unblacklisted user claims the cycle rewards other stakers will be unable to claim their portion of the rewards.

Recommendation: One way of solving it will be to increase the reward token balance of the contract whenever a user, eligible to earn cycle rewards, gets unblacklisted.

Resolution: Acknowledged.

7.4 Informational Findings

[I1] There is no incentive for claiming the protocol's V2 rewards. In the HeliosStaxV2 code comments it is mentioned that there could be incentives for the callers of *claimV2Rewards*, however, currently there isn't. It may be good to add rewards for the callers of the function in order to incentivize users to call it more frequently.